

Proyecto 2 - LogisTEC

*Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica
CE-1103 Algoritmos y Estructuras de Datos I
Profesor: MSc. Andrés Vargas
Semestre I, 2026
Cartago, Costa Rica*

Integrantes:

María José Solano Cordero, Alejandro Salazar Barboza,
Windell Joseph Loría Alfaro, Ian Vargas Corrales
{m.solano.6, a.salazar.7, w.loria.1, i.vargas.3}@estudiantec.cr

Abstract— Este documento presenta el desarrollo de LogisTEC, una aplicación en Java orientada al análisis de distribución logística mediante grafos. El proyecto tuvo como objetivo representar una ciudad como un grafo no dirigido y ponderado, cargar casos desde archivos JSON, visualizar rutas de entrega y aplicar algoritmos de conectividad, caminos mínimos y árboles de expansión mínima. Para ello se implementaron estructuras de datos propias, algoritmos sobre grafos, asignación de paquetes a camiones y heurísticas de planificación de rutas. Como resultado, se obtuvo un sistema funcional capaz de mostrar el grafo, consultar caminos mínimos, comparar algoritmos y generar rutas recomendadas para la distribución de paquetes.

Introducción

LogisTEC es una aplicación de escritorio desarrollada en Java para apoyar la visualización y el análisis de un problema de distribución logística mediante grafos. El sistema representa una ciudad como una red de vértices y aristas, donde los vértices pueden corresponder a intersecciones, puntos de entrega o al depósito principal, mientras que las aristas representan las calles o conexiones entre dichos puntos. La aplicación permite cargar casos de prueba desde archivos en formato JSON, visualizar gráficamente el grafo de la ciudad, identificar el depósito, observar las rutas asignadas a los camiones, revisar paquetes pendientes o no asignados y consultar caminos mínimos

entre puntos específicos. De esta manera, el usuario puede comprender de forma visual cómo se organiza la distribución de paquetes dentro del escenario. Este manual está dirigido al usuario final del sistema. Por esta razón, se enfoca en explicar los requisitos necesarios para ejecutar la aplicación, la forma correcta de cargar casos JSON, la interpretación de la interfaz gráfica y las funciones principales disponibles durante el uso del programa. Los detalles internos de implementación, estructuras de datos y algoritmos se desarrollan en las secciones técnicas posteriores de la documentación.

Manual de usuario

Requerimientos del sistema

Para ejecutar correctamente LogisTEC, se recomienda contar con los siguientes requisitos:

Categoría	Requerimiento
Lenguaje	Java
Versión de JDK utilizada	JDK 21.0.10
Interfaz gráfica	JavaFX 21.0.10
Lectura de JSON	Gson 2.10.1
IDE recomendado	IntelliJ IDEA
Sistema operativo	Windows, Linux o macOS compatible con Java
Memoria RAM recomendada	4 GB o superior
Espacio disponible	Al menos 100 MB libres

El proyecto fue probado desde IntelliJ IDEA. Sin embargo, puede ejecutarse desde otro entorno siempre que el JDK, JavaFX y Gson estén configurados correctamente.

Bibliotecas necesarias para la ejecución

Para que el programa funcione correctamente, deben estar agregadas las siguientes bibliotecas al proyecto:

Biblioteca	Propósito
JavaFX	Permite mostrar la interfaz gráfica del sistema.
Gson	Permite leer archivos JSON y convertir sus datos en objetos utilizados por el programa.

En el proyecto utilizado se observa la biblioteca `gson-2.10.1` y una biblioteca `lib`, donde se incluyen los componentes externos necesarios para la ejecución.

Configuración de JavaFX

Si el programa no ejecuta directamente, se debe revisar que JavaFX esté configurado correctamente en el entorno utilizado. En caso de usar IntelliJ IDEA, se puede verificar desde la opción *File* → *Project Structure* → *Libraries*.

También puede ser necesario revisar la configuración de ejecución del programa y agregar las opciones de máquina virtual correspondientes a JavaFX. Un ejemplo general de configuración es:

```
1 --module-path "RUTA_DE_JAVAFX\lib" --add-modules  
  javafx.controls,javafx.fxml
```

Donde `RUTA_DE_JAVAFX` debe reemplazarse por la ruta real donde se encuentra instalado JavaFX en la computadora. Si JavaFX ya está agregado correctamente como biblioteca del proyecto y el programa ejecuta sin errores, no es necesario modificar esta configuración.

Cómo ejecutar el programa

Para ejecutar LogisTEC, primero se debe abrir el proyecto en un entorno compatible con Java como lo puede ser IntelliJ IDEA, Eclipse, Visual Studio Code (VS) o Apache NetBeans.

Luego, se debe verificar que el JDK, JavaFX y Gson estén correctamente configurados para que la ejecución funcione.

La clase principal del sistema se encuentra en:

```
1 src/app/Main.java
```

Desde esta clase se inicia la ventana principal de la aplicación. Al ejecutarse correctamente, el sistema muestra la pantalla inicial de **LogisTEC**, donde el usuario puede seleccionar un caso JSON de los ya cargados anteriormente, cargar un caso externo que cumpla con la estructura que se solicita y visualizar el grafo.

Pantalla inicial

La pantalla inicial es el primer espacio que observa el usuario al abrir la aplicación. En esta ventana se encuentran los siguientes elementos:

Elemento	Función
Selector de caso JSON	Permite escoger uno de los casos incluidos en el proyecto.
Botón "Cargar caso JSON externo"	Permite cargar un archivo JSON desde la computadora.
Botón "Visualizar grafo"	Abre la visualización gráfica del caso seleccionado.



Figura 1: Pantalla Inicial

Casos disponibles en el sistema

El programa incluye varios casos JSON dentro de la carpeta `data`. Los casos disponibles desde la pantalla inicial son:

Caso	Archivo	Descripción
Caso pequeño	<code>data/caso_pequeno.json</code>	Caso reducido para probar la carga y visualización.
Caso medio	<code>data/caso_medio.json</code>	Caso intermedio con mayor cantidad de datos.
Caso mínimo obligatorio	<code>data/caso_minimo.json</code>	Caso principal con los requisitos mínimos del proyecto.
Caso con paquetes inalcanzables	<code>data/caso_inaccesible.json</code>	Caso para validar destinos no alcanzables desde el depósito.

Para seleccionar un caso, el usuario debe abrir el menú desplegable, escoger el caso que desea visualizar y presionar el botón **Visualizar grafo**. Si no se selecciona ningún caso, el programa muestra un mensaje indicando que primero debe seleccionarse un archivo JSON.

Carga de un JSON externo

Además de los casos incluidos en el proyecto, el sistema permite cargar un archivo JSON externo. Para hacerlo, el usuario debe presionar el botón **Cargar**

caso JSON externo, buscar el archivo en la computadora, seleccionar un archivo con extensión `.json`, confirmar la selección y presionar **Visualizar grafo**.

El archivo externo debe respetar el formato esperado por el programa. Si el archivo está incompleto, contiene datos inválidos o no cumple con la estructura requerida, el sistema mostrará una alerta de error.

Formato esperado del archivo JSON

El archivo JSON debe contener tres secciones principales: **ciudad**, **paquetes** y **camiones**.

La sección **ciudad** contiene los vértices y las aristas del grafo; la sección **paquetes** contiene los paquetes por entregar; y la sección **camiones** contiene la información de la flota disponible.

```

1 {
2   "ciudad": {
3     "vertices": [
4       {
5         "id": "V01",
6         "tipo": "DEPOT",
7         "x": 80,
8         "y": 80
9       },
10      {
11        "id": "V02",
12        "tipo": "INTERSECCION",
13        "x": 180,
14        "y": 70
15      },
16      {
17        "id": "V05",
18        "tipo": "ENTREGA",
19        "x": 480,
20        "y": 90
21      }
22    ],
23    "aristas": [
24      {
25        "u": "V01",
26        "v": "V02",
27        "distancia": 120
28      },
29      {
30        "u": "V02",
31        "v": "V05",
32        "distancia": 140
33      }
34    ]
35  },
36  "paquetes": [
37    {
38      "id": "P01",
39      "destino": "V05",
40      "peso": 8,
41      "prioridad": 1
42    }
43  ],
44  "camiones": [
45    {
46      "id": "C01",
47      "capacidad": 35
48    }
49  ]
50 }
```

Descripción de los campos del JSON

Vértices. Cada vértice representa un punto dentro de la ciudad. El campo `id` corresponde al identificador único del vértice, `tipo` indica si es **DEPOT**, **INTERSECCION** o **ENTREGA**, mientras que `x` y `y` indican la posición utilizada para dibujarlo en la interfaz.

Aristas. Cada arista representa una calle o conexión entre dos vértices. Los campos `u` y `v` indican los vértices conectados, y `distancia` representa el peso de la conexión en metros. El grafo se interpreta como no dirigido, por lo que la conexión puede recorrerse en ambos sentidos.

Paquetes. Cada paquete contiene un `id`, un `destino`, un `peso` en kilogramos y una `prioridad`. La prioridad puede ser 1, 2 o 3, donde 1 representa la mayor prioridad.

Camiones. Cada camión contiene un `id` y una `capacidad`. La capacidad indica el peso máximo que puede transportar. Cada camión inicia y finaliza su recorrido en el depósito.

Pantalla de visualización del grafo

Después de cargar un caso correctamente, se muestra la pantalla de visualización del grafo. Esta pantalla contiene una barra superior con los botones principales, un área central donde se dibuja la ciudad y un panel derecho con la leyenda visual más importante.

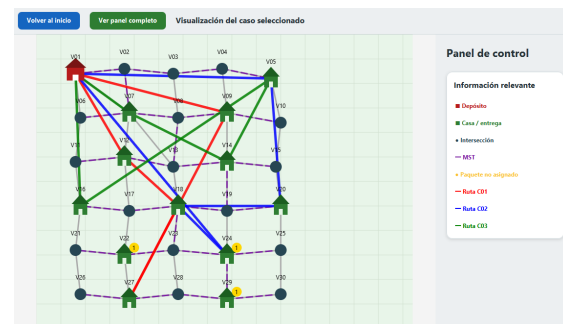


Figura 2: Pantalla principal

Simbología del grafo

La interfaz utiliza colores y figuras para que el usuario pueda interpretar fácilmente el caso cargado.

Elemento visual	Significado
Casa roja	Depósito principal.
Casa verde	Punto de entrega.
Punto azul oscuro	Intersección.
Línea gris	Calle o arista del grafo.
Línea morada punteada	Árbol de expansión mínima, MST.
Círculo amarillo	Paquete pendiente o no asignado.
Línea roja	Ruta del camión C01.
Línea azul	Ruta del camión C02.
Línea verde	Ruta del camión C03.

Esta simbología también aparece en el panel de control para que el usuario pueda consultar el significado de cada color mientras analiza el grafo.

Botones principales de la visualización

En la pantalla de visualización se encuentran dos botones principales. El botón **Volver al inicio** permite regresar a la pantalla principal para seleccionar otro caso JSON, cargar un archivo externo diferente o reiniciar la visualización actual.

El botón **Ver panel completo** abre una ventana adicional con información más detallada del caso cargado, como el resumen general, la consulta de camino mínimo, datos del MST, información de camiones, comparación de heurísticas y paquetes no asignados. En la siguiente imagen, se muestran los botones y el panel de control desplegado.

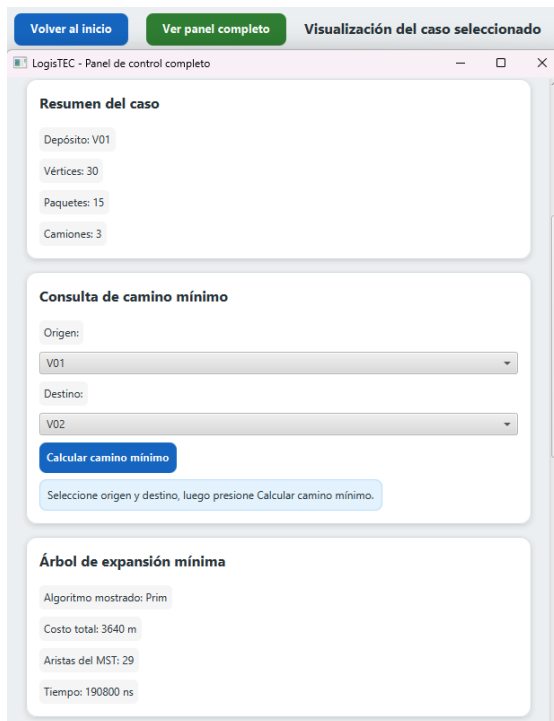


Figura 3: Botones y panel de control de la pantalla principal

Secciones del panel completo

El panel completo está dividido en varias secciones. Entre ellas se encuentran la información relevante, el resumen del caso, la consulta de camino mínimo, la información del árbol de expansión mínima, los datos de camiones, la comparación de heurísticas y los paquetes no asignados. Además, posee una barra de desplazamiento vertical para revisar toda la información disponible.

Consulta de camino mínimo

El sistema permite consultar el camino mínimo entre dos vértices del grafo. Para usar esta función, el usuario debe abrir el **panel completo**, ir a la sección **Consulta de camino mínimo**, seleccionar el vértice de origen, seleccionar el vértice de destino y presionar el botón correspondiente para calcular el camino.

El resultado muestra el orden de los vértices que forman el recorrido, la distancia total en metros o una indicación si no existe camino entre el origen y el destino.

Visualización de MST, rutas y paquetes no asignados

La visualización del grafo permite observar tres resultados importantes del sistema: el MST, las rutas de los camiones y los paquetes no asignados.

El **MST** se muestra con líneas moradas punteadas y representa una forma mínima de conectar los vértices del grafo. Las **rutas de los camiones** se muestran con colores diferentes: rojo para C01, azul para C02 y verde para C03. Cada ruta representa el recorrido recomendado para un camión según los paquetes asignados y las heurísticas aplicadas por el sistema. Los **paquetes no asignados** se representan mediante círculos amarillos; esto puede ocurrir si el destino no es alcanzable desde el depósito o si no existe capacidad suficiente en los camiones disponibles.

Reporte en consola

Al cargar un caso, el sistema también genera información en la consola del entorno de ejecución. La consola puede mostrar mensajes relacionados con paquetes alcanzables e inalcanzables, asignaciones realizadas, rutas calculadas, resultados de los algoritmos y comparaciones de distancias. Esta salida es útil durante la revisión del proyecto porque permite confirmar que el programa está realizando los cálculos correspondientes, además de mostrarlos en la interfaz gráfica.

Errores comunes y posibles soluciones

Situación	Posible causa	Solución
El programa no abre	El JDK no está configurado.	Revisar que el proyecto use JDK 21 o compatible.
Error de JavaFX	JavaFX no está agregado o faltan VM Options.	Agregar JavaFX y revisar la configuración.
Error de Gson	Gson no está agregado.	Agregar <code>gson-2.10.1</code> .
No carga el JSON	Archivo mal formado o incompleto.	Revisar que tenga ciudad, vértices, aristas, paquetes y camiones.
No se muestra ruta	No hay paquetes asignados o destinos alcanzables.	Revisar paquetes, camiones y conexiones.
Paquetes pendientes	Destino inalcanzable o capacidad insuficiente.	Revisar conectividad y capacidad.

Recomendaciones de uso y cierre del programa

Para utilizar correctamente LogisTEC, se recomienda ejecutar el proyecto desde un entorno compatible con Java, verificar que JavaFX y Gson estén agregados, usar primero el **Caso mínimo obligatorio** para comprobar la funcionalidad principal y revisar la consola después de cargar un caso.

Si se carga un JSON externo, se debe comprobar que respete el formato esperado. También se recomienda no eliminar la carpeta de recursos visuales, ya que contiene imágenes utilizadas por la interfaz. Para cerrar el programa, el usuario puede presionar el botón de cierre de la ventana. No es necesario guardar cambios, ya que el sistema trabaja con archivos JSON de entrada y muestra los resultados directamente en pantalla y consola.

Estructuras de datos y grafo base

La base del sistema LogisTEC se construyó a partir de estructuras de datos propias y de una representación de grafo no dirigido y ponderado. Esta sección resume los tipos de datos abstractos implementados, la forma en que se representó el grafo y la relación de estas clases con los algoritmos utilizados posteriormente.

El diseño se organizó principalmente en dos paquetes: **structures** y **graph**. El paquete **structures** contiene las estructuras implementadas manualmente para el núcleo algorítmico. El paquete **graph** contiene las clases que permiten representar la red vial de la ciudad mediante vértices, aristas y listas de adyacencia.

Estructuras de datos implementadas

Las estructuras propias se implementaron para evitar depender de colecciones externas dentro del núcleo del proyecto. Estas estructuras sirven como apoyo para recorridos, caminos mínimos y construcción de árboles de expansión mínima.

Clase	Función principal	Complejidad principal
<code>Node<T></code>	Nodo genérico utilizado por estructuras enlazadas. Guarda un dato y una referencia al siguiente nodo.	$O(1)$
<code>MyLinkedList<T></code>	Lista enlazada simple utilizada para almacenar elementos de forma dinámica, especialmente vecinos del grafo.	$O(n)$ en búsqueda o inserción final; $O(1)$ en operaciones al inicio.
<code>MyQueue<T></code>	Cola genérica con comportamiento FIFO, utilizada para procesar vértices en orden de llegada.	$O(1)$ en enqueue y dequeue.
<code>MyStack<T></code>	Pila genérica con comportamiento LIFO, útil para recorridos o reconstrucción de caminos.	$O(1)$ en push y pop.
<code>PriorityNode<T></code>	Nodo auxiliar para la cola de prioridad. Guarda un dato y su prioridad.	$O(1)$
<code>MyPriorityQueue<T></code>	Cola de prioridad mínima implementada con heap binario. Permite extraer el elemento de menor prioridad.	$O(\log n)$ en insertar y extraer mínimo.
<code>UnionFind</code>	Estructura de conjuntos disjuntos usada para unir componentes y detectar ciclos.	$O(\alpha(n))$ amortizado.

Cuadro 1: Estructuras de datos implementadas en el paquete **structures**.

A nivel interno, **MyLinkedList** se construyó con nodos enlazados mediante una referencia **head** y un contador de tamaño. **MyQueue** utiliza referencias **front** y **rear** para insertar al final y extraer desde el frente en tiempo constante. **MyStack** utiliza una referencia **top**, por lo que las operaciones de apilar y desapilar se realizan sobre la cima. **MyPriorityQueue** se implementó con un arreglo que representa un heap binario mínimo, de modo que el elemento con menor prioridad se mantiene cerca de la raíz. **UnionFind** utiliza arreglos **parent** y **rank**, aplicando compresión de caminos y unión por rango para reducir el costo de las operaciones.

La lista enlazada se utiliza como base para almacenar vecinos dentro del grafo. La cola permite apoyar recorridos como BFS. La pila puede utilizarse en procesos de retroceso o reconstrucción de rutas. La cola de prioridad permite seleccionar elementos de menor costo en algoritmos como Dijkstra y Prim. Finalmente, **UnionFind** permite validar si dos vértices ya pertenecen al mismo conjunto, lo cual es necesario para evitar ciclos en Kruskal.

Representación del grafo

El grafo de LogisTEC se implementó como un grafo no dirigido y ponderado mediante listas de adyacencia. Cada vértice representa un punto de la ciudad y cada arista representa una calle con una distancia asociada. Al ser no dirigido, cada conexión se guarda en ambos sentidos.

Clase	Responsabilidad
Vertex	Representa un punto de la ciudad. Guarda identificador, tipo de vértice y coordenadas para la interfaz gráfica.
Edge	Representa una arista ponderada entre dos vértices. Guarda origen, destino y peso.
AdjacencyNode	Representa un vecino dentro de la lista de adyacencia. Guarda destino y peso de la conexión.
Graph	Administra el grafo completo mediante un arreglo de vértices y un arreglo de listas enlazadas de vecinos.

Cuadro 2: Clases principales del paquete **graph**.

La clase **Graph** mantiene un arreglo de vértices, un arreglo de listas de adyacencia y un contador de vértices agregados. Al agregar una arista entre dos vértices, se valida que ambos existan, que el peso sea positivo y que no exista una conexión duplicada. Luego, la conexión se agrega en ambos sentidos para respetar el comportamiento de un grafo no dirigido.

API principal del grafo

Método	Descripción	Complejidad
<code>addVertex(vertex)</code>	Agrega un vértice si no supera la capacidad y no está repetido.	$O(V)$
<code>addEdge(from,to,weight)</code>	Agrega una arista no dirigida y ponderada.	$O(V + grado)$
<code>getVertexIndex(id)</code>	Busca el índice interno de un vértice.	$O(V)$
<code>getVertex(index)</code>	Retorna un vértice según su posición.	$O(1)$
<code>getVertexById(id)</code>	Retorna un vértice según su identificador.	$O(V)$
<code>getNeighbors(vertexId)</code>	Retorna la lista de vecinos de un vértice.	$O(V)$
<code>getWeight(from,to)</code>	Retorna el peso de una arista directa.	$O(V + grado)$
<code>hasEdge(from,to)</code>	Verifica si existe una conexión directa.	$O(V + grado)$
<code>getVertexCount()</code>	Retorna la cantidad de vértices agregados.	$O(1)$
<code>getEdgeCount()</code>	Retorna la cantidad de aristas únicas del grafo.	$O(V + E)$

Cuadro 3: Operaciones principales de la clase **Graph**.

Donde V representa la cantidad de vértices, E la cantidad de aristas y $grado$ la cantidad de vecinos del vértice consultado.

Justificación de diseño

La representación por listas de adyacencia fue seleccionada porque una red vial normalmente no conecta todos los puntos con todos los demás. Por esta razón, resulta más conveniente almacenar únicamente las conexiones reales de cada vértice en lugar de utilizar una matriz completa de adyacencia.

Además, la separación entre **Vertex**, **Edge**, **AdjacencyNode** y **Graph** permite dividir responsabilidades: los vértices almacenan información de los puntos de la ciudad, las aristas representan conexiones ponderadas, los nodos de adyacencia guardan vecinos directos y el grafo administra la estructura general.

Relación con los algoritmos

Las estructuras de datos y el grafo base sirven como soporte para los algoritmos principales del proyecto. **MyQueue** se relaciona con BFS, **MyStack** con recorridos o reconstrucción de caminos, **MyPriorityQueue** con Dijkstra y Prim, **UnionFind** con Kruskal y **MyLinkedList** con la representación de vecinos del grafo.

De esta forma, el resto del sistema puede ejecutar algoritmos de conectividad, caminos mínimos, árboles de expansión mínima y planificación de rutas sobre una misma representación base de la ciudad.

Algoritmos implementados

1. Búsqueda en Anchura (BFS)

Descripción:

El algoritmo BFS (*Breadth-First Search*) explora los vértices de un grafo por niveles. Visita primero todos los vecinos directos de un nodo antes de pasar a los elementos del siguiente nivel de profundidad. Utiliza una cola (FIFO) para gestionar el orden y un conjunto de visitados para prevenir ciclos.

Pseudocódigo

```
1 Funcion BFS(Grafo grafo, Nodo nodoOrigen)
2   Crear Conjunto visitados
3   Crear Cola cola
4   Agregar nodoOrigen a visitados
5   Encolar nodoOrigen en cola
6   Mientras cola no este vacia Hacer
7     Nodo nodoActual = cola.desencolar()
8     Para Cada AdjacencyNode vecino En grafo.
9       obtenerVecinosDe(nodoActual) Hacer
10        Si vecino.id no esta en visitados
11          Entonces
12            Agregar vecino.id a visitados
13            Encolar grafo.obtenerNodo(vecino.id) en cola
14          Fin Si
15        Fin Para
16      Fin Mientras
17    Fin Funcion
```

Ejemplo Trabajado a Mano

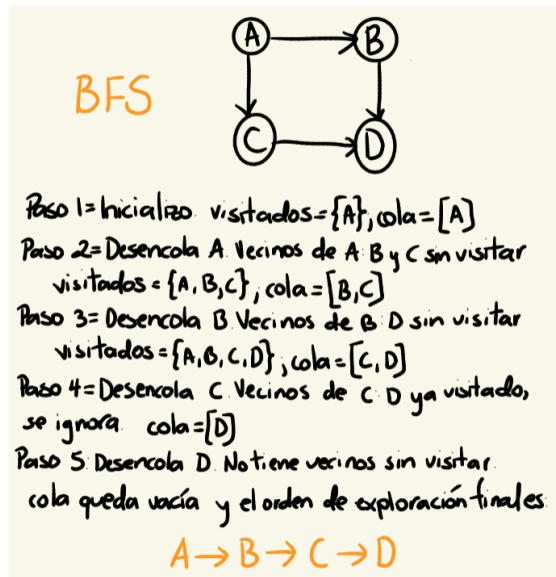


Figura 4: Simulación analítica del recorrido BFS utilizando una cola.

Complejidades

- **Temporal:** $O(V + E)$, donde V es vértices y E aristas.
- **Espacial:** $O(V)$ por la estructura de la cola y visitados.

2. Búsqueda en Profundidad (DFS)

Descripción:

El algoritmo DFS (*Depth-First Search*) recorre los caminos hacia la máxima profundidad de cada rama antes de realizar un proceso de retroceso (*backtracking*). Utiliza una estructura de tipo pila (LIFO) para recordar el camino de retorno.

Pseudocódigo

```

1 Funcion DFS(Grafo grafo, Nodo nodoOrigen)
2   Crear Conjunto visitados
3   Crear Pila pila
4   Empilar nodoOrigen en pila
5   Mientras pila no este vacia Hacer
6     Nodo nodoActual = pila.desapilar()
7     Si nodoActual.id no esta en visitados
8       Entonces
9         Agregar nodoActual.id a visitados
10        Para Cada AdjacencyNode vecino En grafo
11          .obtenerVecinosDe(nodoActual) Hacer
12            Si vecino.id no esta en visitados
13              Entonces
14                Empilar grafo.obtenerNodo(
15                  vecino.id) en pila
16              Fin Si
17            Fin Para
18          Fin Si
19        Fin Mientras
20      Fin Funcion
  
```

Ejemplo Trabajado a Mano

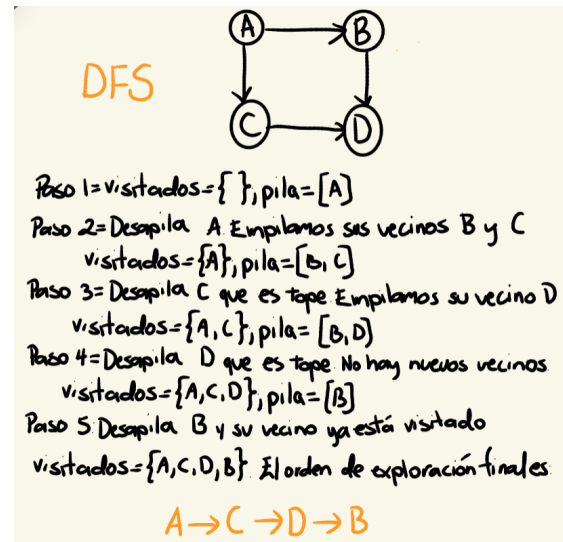


Figura 5: Simulación analítica del recorrido DFS utilizando una pila.

Complejidades

- **Temporal:** $O(V + E)$ al evaluar todas las ramificaciones.
- **Espacial:** $O(V)$ por la profundidad de la pila de llamadas.

3. Algoritmo de Warshall

Descripción:

Se utiliza para encontrar el cierre transitivo de un grafo dirigido. Construye una matriz booleana que indica si existe al menos un camino de cualquier longitud entre cada par de nodos evaluados usando programación dinámica.

Pseudocódigo

```

1 Funcion Warshall(Matriz adyacencia[][][])
2   Int N = adyacencia.longitud
3   Crear Matriz camino[N][N] de booleanos
4   Para i desde 0 hasta N-1 Hacer
5     Para j desde 0 hasta N-1 Hacer
6       camino[i][j] = adyacencia[i][j] > 0 o i == j
7     Fin Para
8   Fin Para
9   Para k desde 0 hasta N-1 Hacer
10    Para i desde 0 hasta N-1 Hacer
11      Para j desde 0 hasta N-1 Hacer
12        camino[i][j] = camino[i][j] o (
13          camino[i][k] y camino[k][j])
14      Fin Para
15    Fin Para
16  Retornar camino
17 Fin Funcion
  
```

Ejemplo Trabajado a Mano

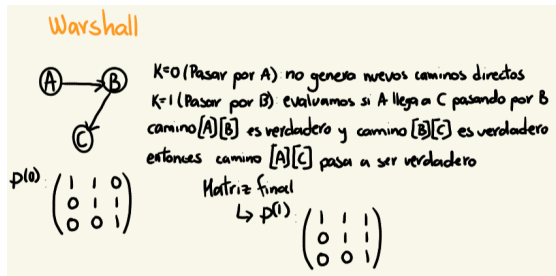


Figura 6: Matrices lógicas booleanas calculadas en la traza de Warshall.

Complejidades

- **Temporal:** $O(V^3)$ debido a los tres bucles independientes anidados.
- **Espacial:** $O(V^2)$ para registrar la matriz de conectividad.

4. Algoritmo de Dijkstra

Descripción:

Resuelve el problema del camino más corto desde un único nodo origen hacia todos los demás nodos en un grafo con pesos no negativos, actualizando las distancias mediante la relajación de aristas.

Pseudocódigo

```

1  Funcion Dijkstra(Grafo grafo, Nodo origen, Nodo
    destino)
2      Crear Diccionario distancias
3      Crear Diccionario padres
4      Crear Conjunto noVisitados
5      Para Cada Nodo n En grafo.obtenerTodosLosNodos
        () Hacer
6          distancias[n.id] = INFINITO
7          padres[n.id] = NULL
8          Agregar n.id a noVisitados
9      Fin Para
10     distancias[origen.id] = 0
11     Mientras noVisitados no este vacio Hacer
12         Nodo u = obtenerNodoMenorDistancia(
            noVisitados, distancias)
13         Si u == NULL o distancias[u.id] == INFINITO
             Entonces
14             Break
15         Fin Si
16         Eliminar u.id de noVisitados
17         Si u.id == destino.id Entonces Break
18         Para Cada AdjacencyNode vecino En grafo.
            obtenerVecinosDe(u) Hacer
19             Int nuevaDistancia = distancias[u.id] +
                vecino.peso
20             Si nuevaDistancia < distancias[vecino.
                id] Entonces
21                 distancias[vecino.id] =
                    nuevaDistancia
22                 padres[vecino.id] = u.id
23         Fin Si
24     Fin Para
25     Fin Mientras

```

```

26     Si distancias[destino.id] == INFINITO Entonces
27         Retornar ShortestPathResult(origen.id,
            destino.id, INFINITO, MyLinkedList(),
            0)
28     Sino
29         MyLinkedList path = reconstruirCamino(
            padres, destino.id)
30         Retornar ShortestPathResult(origen.id,
            destino.id, distancias[destino.id],
            path, path.longitud())
31     Fin Si
32     Fin Funcion

```

Ejemplo Trabajado a Mano

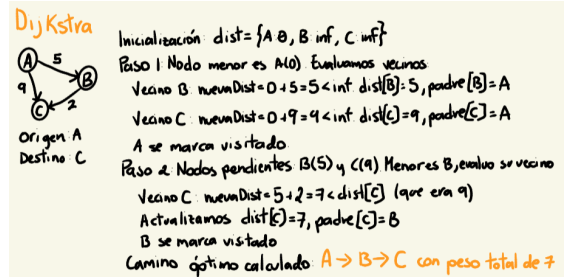


Figura 7: Relajación de aristas y asignación de pesos de Dijkstra.

Complejidades

- **Temporal:** $O(V^2)$ al realizar la búsqueda lineal directa del nodo mínimo pendiente.
- **Espacial:** $O(V)$ para estructurar los vectores de distancias y padres.

5. Algoritmo de Floyd-Warshall

Descripción:

Encuentra los caminos mínimos entre todos los pares de nodos en una única ejecución global. Compara todos los caminos posibles a través de un conjunto creciente de vértices intermediarios.

Pseudocódigo

```

1  Funcion FloydWarshall(Matriz grafoPesos[][] )
2      Int N = grafoPesos.longitud
3      Crear Matriz distancias[N][N]
4      Para i desde 0 hasta N-1 Hacer
5          Para j desde 0 hasta N-1 Hacer
6              distancias[i][j] = grafoPesos[i][j]
7          Fin Para
8      Fin Para
9      Para k desde 0 hasta N-1 Hacer
10         Para i desde 0 hasta N-1 Hacer
11             Para j desde 0 hasta N-1 Hacer
12                 Si distancias[i][k] != INFINITO y
                    distancias[k][j] != INFINITO
13                     Si distancias[i][k] +
                        distancias[k][j] <
                            distancias[i][j] Entonces

```

```

14         distancias[i][j] =
15             distancias[i][k] +
16             distancias[k][j]
17         Fin Si
18     Fin Para
19 Fin Para
20 Retornar distancias
21 Fin Funcion

```

Ejemplo Trabajado a Mano

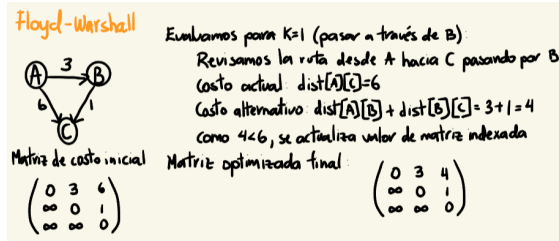


Figura 8: Optimización dinámica de matrices mediante Floyd-Warshall.

Complejidades

- **Temporal:** $O(V^3)$ constante debido a sus tres bucles iterativos globales.
- **Espacial:** $O(V^2)$ para preservar la matriz bidimensional de costos mínimos.

Comparación empírica Prim vs Kruskal

Para comparar empíricamente los algoritmos de Prim y Kruskal se utilizaron tres instancias de prueba con diferentes tamaños: `caso_pequeno.json`, `caso_medio.json` y `caso_minimo.json`. Estas instancias se encuentran en la carpeta `data` del proyecto.

Prim y Kruskal son algoritmos utilizados para obtener un árbol de expansión mínima, conocido como MST por sus siglas en inglés: *Minimum Spanning Tree*. Un MST permite conectar los vértices de un grafo utilizando un conjunto de aristas de costo mínimo, sin formar ciclos.

La comparación se realizó ejecutando directamente los métodos implementados en la clase `MSTAlgorithms`:

- `prim(Graph graph, String startId)`
- `kruskal(Graph graph)`

Para medir los tiempos se creó una clase auxiliar llamada `BenchmarkRunner`. Esta clase ejecutó ambos

algoritmos varias veces, realizó una fase inicial de calentamiento de la JVM y calculó el tiempo promedio con base en `System.nanoTime()`. Además, el benchmark fue ejecutado tres veces y se promediaron esas tres mediciones finales.

Análisis de resultados

En las tres instancias evaluadas, Prim y Kruskal obtuvieron el mismo costo total del MST. Esto confirma que ambas implementaciones generan un resultado equivalente en términos de costo.

La diferencia principal se observa en el tiempo de ejecución. En los tres casos, Prim obtuvo menor tiempo promedio que Kruskal. Esta diferencia se relaciona con la implementación del proyecto: Prim utiliza una cola de prioridad para seleccionar progresivamente la siguiente arista de menor costo, mientras que Kruskal recopila las aristas del grafo, las ordena y luego construye el MST usando Union-Find.

Por lo tanto, aunque ambos algoritmos fueron correctos respecto al costo final, Prim presentó mejor rendimiento temporal en las tres instancias analizadas.

Nota: Los datos detallados de las mediciones y costos numéricos se consolidan en la Tabla 1 al final de esta sección.

Comparación empírica Nearest Neighbor vs MST-Based

Para comparar las heurísticas de planificación de rutas se utilizaron las mismas tres instancias de prueba: `caso_pequeno.json`, `caso_medio.json` y `caso_minimo.json`.

En esta comparación se evaluaron dos métodos:

- **Nearest Neighbor (NN) o Vecino más cercano:** heurística que construye la ruta paso a paso. Desde la ubicación actual, selecciona como siguiente destino el punto no visitado más cercano. Este proceso se repite hasta visitar todos los destinos asignados al camión y regresar al depósito.
- **MST-Based o basado en MST:** heurística que utiliza una estructura de árbol de expansión mínima para ordenar los destinos. En lugar de decidir únicamente por cercanía local, considera una estructura de conexión mínima entre el depósito y los puntos de entrega.

La comparación se realizó después de cargar cada archivo JSON, construir el grafo, asignar los paquetes a los camiones y calcular la matriz de distancias mínimas mediante Floyd-Warshall. Posteriormente, para

cada camión se ejecutaron las dos heurísticas implementadas en `RoutePlanner`:

- `nearestNeighbor(Graph graph, int[] [] distanceMatrix, Truck truck, String depotId)`
- `mstBasedRoute(Graph graph, int[] [] distanceMatrix, Truck truck, String depotId)`

La distancia total de cada instancia corresponde a la suma de las rutas generadas para todos los camiones. El porcentaje de mejora de MST-Based respecto a Nearest Neighbor se calculó mediante:

$$\text{Mejora MST-Based} = \frac{\text{Dist. NN} - \text{Dist. MST}}{\text{Dist. NN}} \times 100$$

Análisis de resultados

En `caso_pequeno.json` y `caso_medio.json`, ambas heurísticas obtuvieron la misma distancia total. Esto indica que, para esas instancias, el orden de visita generado por las dos estrategias produjo rutas equivalentes.

En `caso_minimo.json`, MST-Based obtuvo una distancia total menor. Nearest Neighbor generó una distancia de 7215, mientras que MST-Based obtuvo 6855. Esto representa una mejora aproximada de 4,99 %.

La diferencia se debe a que Nearest Neighbor toma decisiones locales, eligiendo siempre el destino más cercano desde la posición actual. Esto puede funcionar bien en casos pequeños, pero no siempre produce una mejor ruta global. En cambio, MST-Based considera una estructura de conexión mínima entre los destinos, lo cual genera un recorrido más conveniente cuando aumenta la cantidad de puntos por visitar.

A partir de las pruebas realizadas, se concluye que ambas heurísticas son funcionales para generar rutas válidas. Sin embargo, MST-Based presentó una mejor distancia total en la instancia más grande evaluada, mientras que en las dos instancias menores obtuvo el mismo resultado que Nearest Neighbor.

Nota: La distribución del rendimiento de las rutas y camiones evaluados se puede contrastar en la Tabla 2 al final de esta sección.

Resultados obtenidos de las pruebas empíricas

En la tabla 1, **Costo Prim** y **Costo Kruskal** representan el costo total del árbol de expansión mínima generado por cada algoritmo. Las columnas muestran los tiempos promedios en nanosegundos (ns) y milisegundos (ms) para cada archivo evaluado.

Cuadro 4: Comparación empírica entre Prim y Kruskal.

Métrica / Instancia	caso_pequeno.json	caso_medio.json	caso_minimo.json
Vértices	8	15	30
Aristas	12	25	51
Costo Prim	745	1455	3640
Costo Kruskal	745	1455	3640
Prim ns	4833,10	8293,63	22451,10
Kruskal ns	6979,43	16331,30	36715,70
Prim ms	0,004833	0,008294	0,022451
Kruskal ms	0,006979	0,016331	0,036716

En la tabla 2, **Distancia NN** significa distancia total obtenida por la heurística *Nearest Neighbor*. La fila **Distancia MST** representa la distancia total generada por la heurística basada en MST, indicando su respectivo porcentaje de **Mejora** global.

Cuadro 5: Comparación empírica entre Nearest Neighbor y MST-Based.

Métrica / Instancia	caso_pequeno.json	caso_medio.json	caso_minimo.json
V (Vértices)	8	15	30
E (Aristas)	12	25	51
Paquetes	5	10	15
Camiones	2	3	3
Dist. NN	1290	3000	7215
Dist. MST	1290	3000	6855
Mejora	0,00 %	0,00 %	4,99 %
Mejor Heurística	Empate	Empate	MST-Based

Código auxiliar utilizado para las pruebas empíricas

El siguiente código no forma parte del funcionamiento principal del sistema. Fue creado únicamente como clase auxiliar para obtener los datos empíricos de las comparaciones anteriores. Después de realizar las mediciones, puede eliminarse del proyecto sin afectar la funcionalidad principal.

```

1 package app;
2
3 import algorithms.GraphAlgorithms;
4 import algorithms.MSTAlgorithms;
5 import algorithms.MSTResult;
6 import graph.Graph;
7 import io.JsonLoader;
8 import io.LogisticsData;
9 import model.AssignmentResult;
10 import model.RouteResult;
11 import model.Truck;
12 import planner.PackageAssigner;
13 import planner.RoutePlanner;
14
15 public class BenchmarkRunner {
16
17     private static final String[] CASES = {
18         "data/caso_pequeno.json",
19         "data/caso_medio.json",
20         "data/caso_minimo.json"
21     };
22
23     private static final int WARMUP_RUNS = 100;
24     private static final int MEASURE_RUNS = 1000;

```


ciones no impiden el funcionamiento general del programa, pero sí representan aspectos que podrían mejorarse en versiones futuras.

Dependencia de la configuración local

Una de las principales limitaciones del sistema es que su ejecución depende de una correcta configuración local del entorno de desarrollo. El proyecto fue probado principalmente desde IntelliJ IDEA utilizando JDK 21.0.10, JavaFX 21.0.10 y Gson 2.10.1. Por esta razón, si el proyecto se abre en otra computadora o en otro entorno de desarrollo, puede ser necesario volver a configurar manualmente las bibliotecas externas.

Esta limitación se debe a que JavaFX no siempre viene incluido directamente dentro del JDK, por lo que debe agregarse como biblioteca externa. De igual forma, Gson debe estar correctamente asociado al proyecto para que la lectura de archivos JSON funcione. Si estas dependencias no están configuradas, el sistema puede compilar con errores o mostrar mensajes relacionados con componentes faltantes en tiempo de ejecución.

Formato JSON estricto

El sistema espera que los archivos JSON respeten una estructura específica. Cada caso debe incluir las secciones de ciudad, vértices, aristas, paquetes y camiones. Además, los identificadores de los vértices usados en aristas y paquetes deben coincidir con vértices existentes dentro del grafo.

Esta validación permite evitar inconsistencias durante la planificación logística, pero también implica que un archivo JSON con campos mal escritos, datos faltantes o referencias incorrectas no podrá ser cargado correctamente. Actualmente, el sistema puede detectar errores generales de carga, pero no cuenta con un editor visual de JSON ni con una herramienta que corrija automáticamente los archivos mal formados.

Visualización dependiente de coordenadas

La interfaz gráfica utiliza las coordenadas x y y definidas en el archivo JSON para dibujar los vértices en pantalla. Esto permite controlar manualmente la ubicación de cada punto dentro del grafo, pero también genera una limitación: si las coordenadas no están bien distribuidas, algunos vértices, etiquetas o aristas pueden quedar muy cerca entre sí.

En el caso mínimo obligatorio, la distribución fue preparada para que el grafo fuera interpretable durante la defensa. Sin embargo, para casos más grandes o cargados externamente, podría ser necesario ajustar

las coordenadas manualmente para mejorar la claridad visual.

Interfaz gráfica básica

La interfaz gráfica permite visualizar el grafo, el depósito, las rutas, el MST y los paquetes pendientes. Sin embargo, se mantiene como una interfaz básica orientada a la interpretación del resultado durante la revisión del proyecto. A pesar de esta limitación, la interfaz cumple con el propósito principal de representar visualmente la solución generada por el sistema y facilitar la explicación de su funcionamiento.

Heurísticas de ruteo no exactas

Para la planificación de rutas se utilizan heurísticas como Nearest Neighbor y MST-Based. Estas técnicas permiten obtener rutas válidas y razonables en un tiempo aceptable, pero no garantizan necesariamente la ruta óptima global para todos los casos posibles.

Esto se debe a que la planificación de rutas con múltiples paradas se relaciona con el problema del agente viajero, el cual no suele resolverse mediante fuerza bruta en instancias grandes debido a su alto costo computacional. Por esta razón, el proyecto utiliza heurísticas que buscan una buena solución, aunque no necesariamente la mejor solución matemática posible.

Mediciones empíricas sujetas al entorno

Los tiempos utilizados para comparar Prim contra Kruskal y Nearest Neighbor contra MST-Based son mediciones empíricas obtenidas durante la ejecución del programa. Estos tiempos pueden variar según la computadora utilizada, la carga del sistema operativo, la versión de Java, la cantidad de procesos activos y el comportamiento interno de la JVM.

Por esta razón, los resultados de tiempo deben interpretarse como una referencia experimental dentro del entorno de prueba utilizado, no como valores absolutos universales.

Reporte principalmente visual y en consola

El sistema muestra información relevante en la interfaz gráfica y en la consola del entorno de ejecución. Sin embargo, una limitación actual es que no genera automáticamente un archivo externo con el reporte completo en formato PDF, CSV o TXT. Esto significa que los resultados deben ser consultados directamente desde la ventana del programa o desde la consola.

Resumen de limitaciones

Aunque no afectan la funcionalidad principal, estas limitaciones representan oportunidades de mejora, las principales limitaciones del sistema son:

- Dependencia de una correcta configuración de JDK, JavaFX y Gson.
- Necesidad de que los archivos JSON respeten estrictamente la estructura esperada.
- Visualización dependiente de las coordenadas definidas en el JSON.
- Interfaz gráfica funcional, pero sin herramientas avanzadas de edición o navegación.
- Uso de heurísticas que generan buenas rutas, pero no garantizan siempre la solución óptima global.
- Tiempos empíricos dependientes del entorno de ejecución.
- Ausencia de exportación automática del reporte final a un archivo externo.

Actividades por estudiante

El desarrollo del proyecto se ejecutó de forma progresiva utilizando Git bajo un flujo de ramas individuales y Pull Requests para mantener un historial organizado. El proceso inició con la creación del repositorio y el diseño de las estructuras de datos nativas (restringiendo el uso de `java.util`), para luego implementar los componentes algorítmicos esenciales de conectividad, caminos mínimos y planificación logística—incluyendo la asignación de paquetes, el ruteo de camiones y la gestión de bosques mínimos para manejar grafos no conexos según las restricciones del enunciado. Finalmente, se consolidó la integración general mediante la carga de archivos JSON, el refinamiento de la interfaz gráfica y la creación de casos de prueba controlados, registrando en la bitácora una estimación de horas basada estrictamente en el historial de commits, merges y tareas de documentación completadas.

Cuadro 6: Timesheet general por estudiante

Integrante	Fechas de trabajo	Actividades realizadas	Horas
Ian	25/05/2026 08/06/2026 10/06/2026 11/06/2026	Implementación de estructuras base del proyecto, incluyendo TDAs, cola de prioridad, Union-Find y grafo ponderado. Posteriormente, se mejoró el grafo base para evitar aristas duplicadas y fortalecer la consistencia interna. Redacción y preparación de la documentación técnica del sistema.	17
María	28/05/2026 08/06/2026 09/06/2026 10/06/2026 11/06/2026 12/06/2026	Implementación de algoritmos de conectividad y rutas (Dijkstra origen-destino), validaciones, ajustes visuales de la interfaz y unificación de estructuras para caminos mínimos. Desarrollo, revisión y edición del reporte final y documentación del proyecto.	20
Windell	01/06/2026 04/06/2026 10/06/2026 11/06/2026	Implementación de funcionalidades de MST, asignación de paquetes y planificación de rutas. Actualizaciones al README, corrección de bosques mínimos para casos no conexos y limpieza de código. Elaboración de la documentación e informes de pruebas.	18
Alejandro	24/05/2026 03/06/2026 08/06/2026 10/06/2026 11/06/2026 12/06/2026	Creación inicial del proyecto, carga de datos JSON, mejoras en la interfaz gráfica, visualización de caminos mínimos, configuración de archivos base (<code>.gitignore</code>) y casos de prueba. Redacción de secciones de la documentación final y estructuración del reporte.	20
Total	—	Horas estimadas de trabajo del equipo	75

Al final de este documento se encuentra el link a la bitácora realizada en Microsoft Excel.

Problemas encontrados

Durante el desarrollo se presentaron distintos problemas técnicos relacionados con la integración de estructuras de datos, algoritmos de grafos, lectura de archivos JSON, configuración de bibliotecas externas e interfaz gráfica. En esta sección se describen los principales problemas encontrados, los intentos realizados para resolverlos, la solución aplicada, conclusiones obtenidas y la bibliografía o documentación consultada.

Problema 1: Configuración de JavaFX en diferentes computadoras

Descripción del problema. Uno de los principales problemas encontrados fue la configuración de JavaFX al ejecutar el proyecto en diferentes computadoras. Aunque el código estaba correctamente implementado, en algunos entornos aparecían errores indicando que los componentes de JavaFX no estaban disponibles o que faltaban módulos necesarios para ejecutar la interfaz gráfica.

Este problema se presentó porque JavaFX no siempre viene incluido directamente dentro del JDK utilizado. Además, la ruta de instalación de JavaFX puede variar entre computadoras, lo cual impide que una misma configuración local funcione para todos los integrantes del equipo.

Intentos realizados. Inicialmente se intentó ejecutar el proyecto directamente desde el IDE, suponiendo que las bibliotecas externas ya estaban correctamente enlazadas. Posteriormente, se revisó la estructura del proyecto para confirmar si JavaFX aparecía dentro de las bibliotecas configuradas y se probaron distintas rutas en las opciones de ejecución y se revisó si el problema estaba relacionado con el JDK o con la configuración de módulos.

Solución aplicada. Consistió en configurar JavaFX como una biblioteca externa del proyecto y, cuando fue necesario, agregar las opciones de máquina virtual correspondientes:

```
1 --module-path "RUTA_DE_JAVAFX\lib" --add-modules  
   javafx.controls,javafx.fxml
```

Además, se recomendó que cada integrante configurara JavaFX localmente en su computadora, evitando subir rutas absolutas al repositorio para no generar conflictos entre equipos.

Conclusión. Este problema permitió comprender la importancia de separar el código fuente de las configuraciones locales del entorno. Y evidenció que, cuando se trabaja con bibliotecas gráficas externas, no basta con que el código esté correcto; además, el entorno de ejecución debe estar correctamente preparado.

Bibliografía consultada. Se consultó la documentación oficial de JavaFX y recursos relacionados con la configuración de JavaFX en IntelliJ IDEA, especialmente para comprender el uso de `--module-path` y `--add-modules`.

Problema 2: Lectura y validación de archivos JSON

Descripción del problema. Otro problema importante fue asegurar que la información cargada de archivos JSON se transformara correctamente en los objetos internos del programa. El archivo debía representar la ciudad, los vértices, las aristas, los paquetes y los camiones. Si alguno de estos elementos estaba incompleto o mal referenciado, el sistema podía fallar al construir el grafo o al asignar paquetes. El mayor riesgo era que el programa intentara crear rutas o calcular algoritmos usando vértices inexistentes, distancias inválidas o paquetes con destinos no definidos.

Intentos realizados. Primero se probaron archivos JSON pequeños para verificar que la estructura básica pudiera leerse correctamente. Luego se realizaron pruebas con casos más grandes, incluyendo el caso mínimo obligatorio. También se construyó un caso con paquetes inalcanzables para validar que el sistema pudiera detectar destinos que no podían alcanzarse desde el depósito. Durante estas pruebas se revisó la correspondencia entre los identificadores de vértices, las conexiones definidas en las aristas y los destinos de los paquetes.

Solución aplicada. Se implementó una clase encargada de cargar los datos desde JSON y convertirlos en una estructura interna utilizable por el programa. Además, se definió un formato esperado para los archivos JSON y se agregaron validaciones para reducir errores durante la carga. El sistema verifica que existan las secciones principales, que los vértices tengan identificadores válidos, que las aristas conecten vértices existentes, que exista un depósito y que los paquetes hagan referencia a destinos dentro del grafo.

Conclusión. La lectura de JSON no debe verse únicamente como una operación de entrada de datos, sino como una etapa crítica de validación del sistema. Un error en el archivo de entrada puede afectar todos los procesos posteriores. Por esta razón, documentar claramente el formato JSON esperado fue fundamental para facilitar el uso correcto del programa.

Bibliografía consultada. Se consultó documentación de Gson para el parseo de archivos JSON en Java, además del enunciado del proyecto, donde indicaba que el protocolo de configuración debía documentarse formalmente mediante un esquema o ejemplo anotado.

Problema 3: Integración entre algoritmos y modelo de datos

Descripción del problema. Una dificultad relevante fue integrar los algoritmos de grafos con el modelo logístico del proyecto. Los algoritmos como Warshall, Dijkstra, Floyd-Warshall, Prim y Kruskal trabajan con estructuras propias de grafos, mientras que el problema logístico requiere interpretar esos resultados en términos de paquetes, camiones, rutas, destinos alcanzables y distancias recorridas.

Esto generó un reto de integración, ya que no bastaba con que cada algoritmo funcionara individualmente; sus resultados debían conectarse de forma coherente con el flujo general del sistema.

Intentos realizados. Se realizaron pruebas separadas con grafos pequeños para validar los algoritmos de manera individual. Posteriormente, se integraron los algoritmos al flujo principal del proyecto, empezando por la carga del JSON, la construcción del grafo y la validación de alcanzabilidad. Luego se incorporaron los cálculos de caminos mínimos, MST, asignación de paquetes y planificación de rutas. También se compararon los resultados obtenidos por los algoritmos con casos resueltos manualmente, especialmente para detectar errores en distancias, caminos reconstruidos o paquetes no alcanzables.

Solución aplicada. La solución consistió en organizar el proyecto por responsabilidades. Las clases del paquete de grafos se encargan de representar vértices y aristas; las clases de algoritmos se encargan de los cálculos; las clases del modelo representan paquetes, camiones y resultados; y las clases de planificación conectan los resultados algorítmicos con la lógica logística. Esta separación permitió que los algoritmos pudieran reutilizarse dentro de la interfaz gráfica y dentro de las pruebas empíricas sin mezclar directamente la lógica visual con la lógica algorítmica.

Conclusión. La integración demostró que un proyecto no depende únicamente de implementar correctamente los algoritmos, sino de diseñar una comunicación clara entre las diferentes partes del sistema. Separar responsabilidades facilitó la depuración, redujo errores y permitió que la interfaz mostrara resultados coherentes con los cálculos internos.

Bibliografía consultada. Se consultó el material de la Lección 06 del curso sobre grafos y las referencias recomendadas en el enunciado, especialmente para reforzar el uso de algoritmos clásicos y su aplicación dentro de un problema real.

Problema 4: Restricción de no utilizar colecciones de java.util en el núcleo algorítmico

Descripción del problema. El enunciado del proyecto indicaba que cualquier estructura de datos distinta de un arreglo nativo debía ser implementada por el equipo. Esto significó que no se podían utilizar estructuras de la colección `java.util` dentro del núcleo algorítmico. Esta restricción aumentó la complejidad del desarrollo, ya que algoritmos como Dijkstra, Prim y Kruskal normalmente se implementan con apoyo de estructuras ya disponibles en bibliotecas estándar.

Intentos realizados. Primero se identificaron las estructuras necesarias para el proyecto: lista enlazada, cola, pila, cola de prioridad y union-find. Luego se revisó qué partes del código podían depender de estas estructuras y cuáles debían mantenerse alejadas de colecciones prohibidas. También se revisaron implementaciones para evitar que el núcleo algorítmico dependiera accidentalmente de estructuras no permitidas.

Solución aplicada. Se implementaron TDAs propios para cubrir las necesidades principales del sistema. La lista enlazada se utilizó para almacenar colecciones dinámicas, la cola se utilizó en BFS, la pila en DFS, la cola de prioridad en algoritmos que requieren selección por menor costo y Union-Find en Kruskal para detectar ciclos. De esta forma, el proyecto mantuvo la lógica principal alineada con la restricción del enunciado y reforzó la aplicación práctica de estructuras de datos estudiadas en el curso.

Conclusión. Aunque implementar estructuras propias requiere más trabajo que utilizar colecciones ya existentes, esta decisión permitió comprender mejor cómo funcionan internamente los algoritmos. Además, reforzó la relación entre estructuras de datos y eficiencia algorítmica.

Bibliografía consultada. Se consultó el material del curso sobre TDAs y grafos, además de material en línea que permitieron una mejor orientación.

Problema 5: Representación visual clara del grafo y sus resultados

Descripción del problema. Una vez integrados los cálculos principales, surgió el reto de representar los resultados de forma clara en la interfaz gráfica. El sistema debía mostrar vértices, aristas, depósito, puntos de entrega, MST, rutas de camiones y paquetes no asignados sin que la pantalla se volviera confusa. Este problema fue evidente en el caso mínimo obligatorio, debido a que contiene una mayor cantidad de vértices y aristas. Al aumentar los elementos, también aumenta la posibilidad de cruces de líneas, etiquetas sobrepuestas o dificultad para interpretar los colores.

Intentos realizados. Se probaron diferentes formas de representar los vértices y las rutas. También se trabajó en una leyenda visual para explicar el significado de los colores y símbolos. Además, se agregó un panel de control para separar la información detallada de la visualización principal, evitando saturar la pantalla. Se realizaron ajustes en la ubicación de los elementos, tamaños, colores y distribución del panel lateral para que la información fuera más entendible.

Solución aplicada. La solución fue dividir la información visual en dos niveles. En la pantalla principal se muestra el grafo con una leyenda resumida, mientras que en el panel completo se presenta información más detallada del caso, como resumen general, consulta de camino mínimo, MST, comparación de heurísticas, camiones y paquetes no asignados. También se utilizaron colores diferentes para las rutas de los camiones y una simbología específica para depósito, entregas, intersecciones, MST y paquetes pendientes.

Conclusión. La visualización de grafos no solo depende de dibujar nodos y aristas, sino de organizar la información para que el usuario pueda interpretarla. Separar la vista principal del panel detallado mejoró la claridad de la interfaz y permitió que la aplicación fuera más útil.

Bibliografía consultada. Se consultó documentación de JavaFX relacionada con construcción de interfaces gráficas, manejo de paneles, botones, ventanas y componentes visuales.

Problema 6: Medición empírica de tiempos

Descripción del problema. Para las comparaciones empíricas era necesario medir los tiempos de ejecución de los algoritmos Prim, Kruskal, Nearest Neighbor y MST-Based. Sin embargo, medir tiempos en Java puede producir resultados variables, especialmente en ejecuciones muy pequeñas, debido al comportamiento de la JVM, el calentamiento del compilador y la carga del sistema operativo.

Intentos realizados. Inicialmente se realizaron mediciones directas de una sola ejecución, pero los resultados podían variar mucho. Luego se decidió ejecutar los algoritmos varias veces y trabajar con promedios. También se agregó una fase de calentamiento para reducir el impacto inicial de la JVM.

Solución aplicada. Se creó una clase auxiliar de pruebas llamada **BenchmarkRunner** que ejecuta los algoritmos en los tres casos seleccionados, realiza una fase de calentamiento, mide múltiples veces y calcula tiempos promedio con `System.nanoTime()`. El código auxiliar no forma parte de la funcionalidad principal, sino que fue utilizado como herramienta de medición para construir las tablas de comparación empírica.

Conclusión. La medición empírica requiere cuidado, ya que una sola ejecución no siempre representa adecuadamente el comportamiento de un algoritmo. Usar repeticiones, calentamiento y promedios permitió obtener datos más estables para la documentación.

Bibliografía consultada. Se consultó la documentación oficial de Java sobre `System.nanoTime()` y referencias generales sobre medición de rendimiento en programas Java.

Conclusión general de los problemas encontrados

Los problemas encontrados durante el desarrollo permitieron mejorar tanto el sistema como la comprensión del proyecto. La configuración de dependencias, la lectura de JSON, la integración de algoritmos, la restricción de estructuras propias, la visualización gráfica y la medición empírica fueron retos que obligaron al equipo a revisar el diseño y fortalecer la organización del código. En general, las soluciones aplicadas permitieron obtener un sistema funcional, integrado y coherente con los objetivos del proyecto. Además, cada problema aportó aprendizajes importantes sobre desarrollo en Java, trabajo con grafos, planificación logística, interfaz gráfica y pruebas empíricas.

Conclusiones

El proyecto LogisTEC permitió modelar un problema de distribución logística mediante un grafo no dirigido y ponderado, donde los vértices representan puntos de la ciudad y las aristas representan calles con una distancia asociada. Esta representación permitió integrar de forma ordenada la carga de datos, visualización del mapa, análisis de conectividad, cálculo de caminos mínimos y planificación de rutas.

La implementación de estructuras de datos propias fue fundamental para el desarrollo. La lista enlazada permitió almacenar las listas de adyacencia del grafo, la cola facilitó los recorridos por amplitud, la pila apoyó procesos de recorrido o reconstrucción, la cola de prioridad permitió seleccionar elementos de menor costo en algoritmos como Dijkstra y Prim, y Union-Find permitió detectar ciclos durante la construcción del árbol de expansión mínima con Kruskal.

Los algoritmos implementados permitieron cubrir diferentes necesidades del problema. Warshall permitió validar la alcanzabilidad de destinos desde el depósito, Dijkstra permitió consultar caminos mínimos entre dos puntos específicos, Floyd-Warshall generó una matriz de distancias mínimas entre todos los pares de vértices, y Prim y Kruskal permitieron obtener y comparar árboles de expansión mínima.

Las pruebas empíricas mostraron que Prim y Kruskal generaron el mismo costo total del MST en las instancias evaluadas, lo cual confirma la consistencia de ambas implementaciones. Sin embargo, Prim presentó menor tiempo promedio en las pruebas realizadas, debido a la forma en que selecciona progresivamente las aristas candidatas mediante una cola de prioridad.

En la comparación de rutas, las heurísticas Nearest Neighbor y MST-Based produjeron resultados equivalentes en los casos pequeño y medio. En el caso mínimo obligatorio, MST-Based obtuvo una distancia total menor, lo que evidencia que considerar una estructura global de conexión mejora la planificación cuando aumenta la cantidad de puntos por visitar.

En general, el sistema cumplió con los objetivos planteados, ya que permite cargar casos JSON, construir el grafo, asignar paquetes, calcular rutas, visualizar resultados y generar reportes útiles para analizar el comportamiento de los algoritmos aplicados al problema logístico.

Recomendaciones

Se recomienda mantener una estructura modular en futuras mejoras, separando claramente las responsabilidades de cada paquete. Esta organización facilita la corrección de errores, la incorporación de nuevas funcionalidades y la comprensión general del código.

Para mejorar el rendimiento del grafo, podría implementarse una forma más eficiente de buscar vértices por identificador, siempre respetando las restricciones del proyecto. Actualmente, varias búsquedas se realizan de forma lineal, lo cual es aceptable para los casos evaluados, pero podría afectar el rendimiento en instancias con un mayor tamaño.

Además se recomienda optimizar la lista enlazada incorporando una referencia al último nodo, así la inserción al final podría realizarse en tiempo constante, en lugar de requerir un recorrido completo de la lista.

En la cola de prioridad, una posible mejora sería reducir el costo de la operación decreasePriority. En el código actual, primero debe buscar el elemento dentro del heap, lo cual tiene costo lineal. Una versión futura podría incorporar una estructura auxiliar permitida para localizar elementos con mayor rapidez.

Para las pruebas empíricas, se recomienda aumentar la cantidad de casos evaluados y utilizar instancias con diferentes densidades de aristas, cantidades de paquetes y capacidades de camiones. Esto permitiría observar con mayor claridad el comportamiento de los algoritmos ante escenarios más variados.

Finalmente, se recomienda conservar el uso de archivos JSON externos como mecanismo de prueba,

ya que permite validar el sistema con nuevos escenarios sin modificar el código fuente. Esto facilita la evaluación del proyecto y permite comprobar que la aplicación funciona correctamente con distintas configuraciones de ciudad, paquetes y camiones.

Enlaces del proyecto

A continuación, se presentan enlaces del repositorio que contiene el historial de desarrollo, código fuente y archivos asociados al sistema y la bitácora en formato Excel con el detalle de actividades y horas de trabajo del equipo para este proyecto.

- **Repositorio del proyecto:** [Repositorio LogísTEC en GitHub](#)
- **Bitácora en Excel:** [Bitácora LogísTEC en Excel](#)

Bibliografía

- Baeldung. (s. f.). *Kruskal's vs Prim's algorithm*. Baeldung on Computer Science. <https://www.baeldung.com/cs/kruskals-vs-prim-s-algorithm>
- Google. (2026). *Gemini* [Modelo de lenguaje grande]. <https://gemini.google.com/>
- NetworkLessons. (s. f.). *Multiple Spanning Tree (MST)*. NetworkLessons.com. <https://networklessons.com/spanning-tree/multiple-spanning-tree-mst>
- OpenAI. (2026). *ChatGPT* [Modelo de lenguaje grande]. <https://chatgpt.com/>
- Singh, A. (s. f.). *Floyd-Warshall algorithm: Why it is used in networking?* Medium. <https://medium.com/@singhatul1155/depths-of-the-floyd-warshall-algorithm-most-effecient-for-shortest-path-ac790d888c07>
- Universidad Americana de Europa. (s. f.). *Qué son los grafos*. UNADE. <https://unade.edu.mx/que-son-los-grafos/>
- W3Schools. (s. f.). *DSA Dijkstra's algorithm*. W3Schools. https://www.w3schools.com/dsa/dsa_algo_graphs_dijkstra.php
- W3Schools. (s. f.). *Java data structures*. W3Schools. https://www-w3schools-com.translate.goog/java/java_data_structures.asp?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc